

Threads & Processes

`mano@cs.auckland.ac.nz`

Processes *in cpu*

Process

Registers

store some of value used a lot of time.

Process

Registers

Memory

编译程序分配

Static

执行程序空间

Dynamic

Code

Stack

function not know how much space.

may overflow

above Dynamic area

Memory

Static

Dynamic

Code

Stack

Top

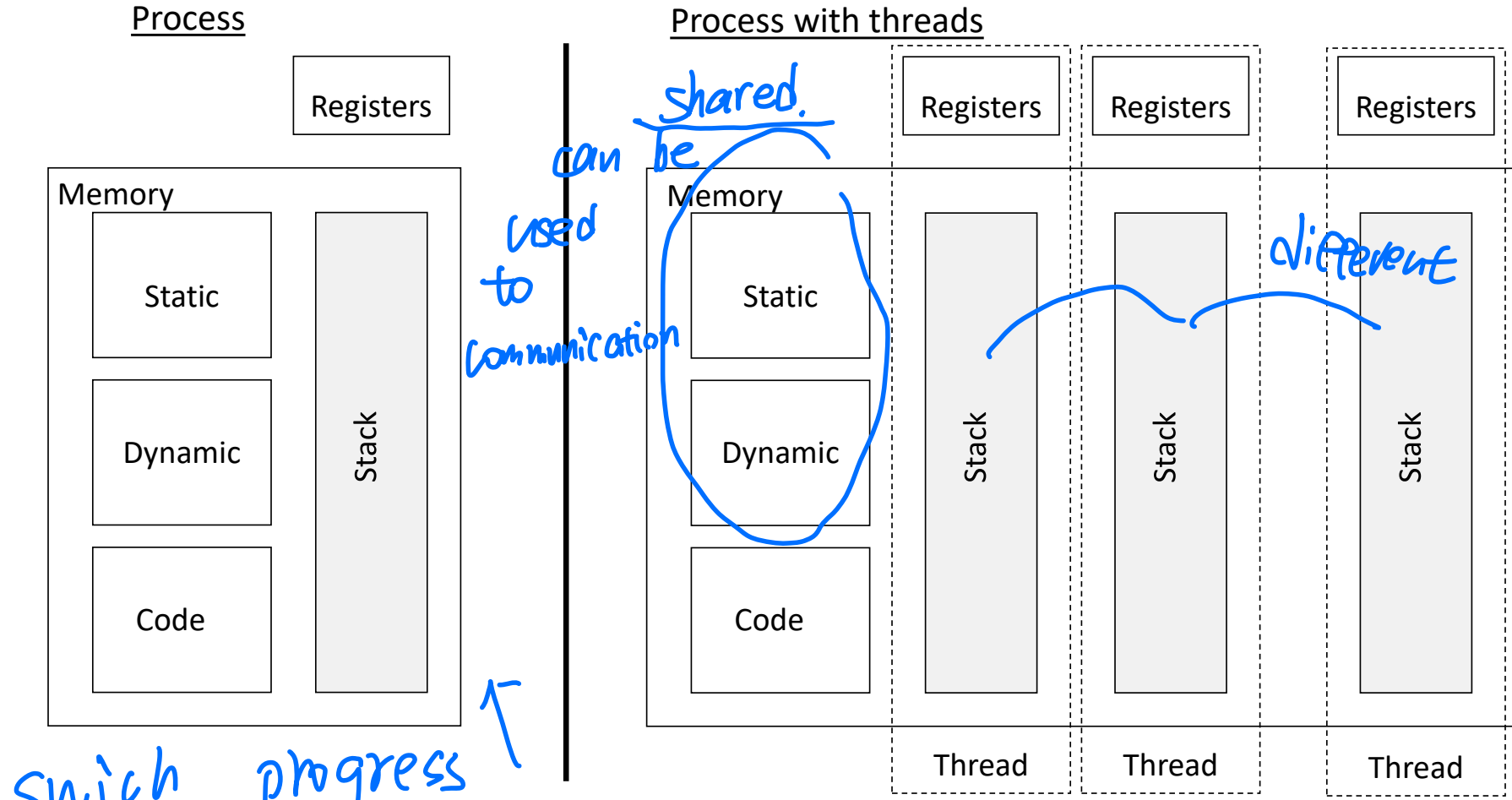
space

*can up
扩大动态区域*

bottom.

eg. [new], not know space

Process vs. Threads



Threads 线程

- A thread is a single, sequential flow of control within a program. Within each thread, there is a single point of execution.
- Threads execute concurrently.
- Most traditional programs consist of a single thread.
- Threads execute within (and share) a single address space.
- Synchronization elements ensure proper memory access.

Advantages of using Threads

- Improve performance
 - Multiprocessors: threads may run on separate processors concurrently.
 - Uniprocessors: threads permit overlapping of slow operations (such as I/O) with computation.

Advantages of using Threads

- Use the natural parallelism in applications.
 - E.g., User interaction (slow) can be a thread.
- Multiple threads allow a server to handle clients' requests in parallel instead of artificially serializing them (e.g., the historical Unix *login* process) or creating one server process per client at great expense (e.g., the historical *httpd*).

Shared Resources

- Address space, shared storage
- The process ID, parent process ID, process group ID
- User ids: real and effective.
- Group ids: real and effective.
- Supplementary group ids
- Current working directory, root directory
- File-mode creation mask
- File descriptor table
- Signal handlers
- Timers

Private Resources

- A unique thread identifier
- Resources required to support a flow of control (such as a stack)
- C/Unix `errno`
- Thread-specific key/value bindings
- Per-thread cancellation handlers
- Per-thread signal masks

Thread Operations

- Creation
- Execution
- Termination
- Cancellation
- Deletion
- Wait for termination
- Synchronization

Shared Resources

bank
5 -2 ✓
-2+5 x

- Threads sharing resources should take care not to result in a *race condition* – behaviour that is time-dependent.
 - A race condition is a concurrency error that occurs when two or more processes or threads access shared resources simultaneously, and the outcome of the program depends on the order in which these processes execute. This leads to unpredictable and potentially incorrect results because the timing of events can be unpredictable, and the program's behavior is not deterministic.
- This is avoided by granting each thread exclusive access to shared resources at any given time.

use locks

Sharing Resources



Agree/Disagree?

Every toilet should have a lock to prevent concurrent access.

A toilet should not have two locks.

Two toilets should not share a single lock.

A toilet and a shower should not share a single lock.

If a toilet is locked, wait for your turn. (busy waiting is waste of time)

Let the user of the toilet, call you out when they have finished using it – this allows you to do other useful things than busy waiting.

Toilet = shared resource

Shower = another shared resource

Mutex 互斥锁

- A mutex is used by multiple threads to ensure the integrity of a shared object that they access by allowing only one thread to access it at a time.
一个线程.
- A mutex has two states, locked and unlocked.
- Each thread locks a mutex before it accesses the shared object and unlocks the mutex when it is finished accessing that object.
- If the mutex is locked by another thread, the thread requesting the lock waits for the mutex to be unlocked.



Mutex

- For each shared object, all threads accessing that data must use the same mutex.
- Each mutex must be initialized before use.

初始化

用前符号

```
mutex m = new mutex();  
m.lock();  
m.unlock();
```



Condition Variables

- A condition variable allows a thread to wait (i.e., block its own execution) until a condition is met (some shared data reaches a particular state).
 - A mutex, on the other hand, allows a thread to wait until a lock is available.
- A condition variable is a synchronization object used in conjunction with a mutex.

Condition Variables

- A mutex controls access to shared data; a condition variable allows threads to wait for that data to enter a defined state.
- The state is defined by a Boolean expression called a predicate.
- A predicate may be a Boolean variable in the shared data or the predicate may be indirect. E.g., testing whether a counter has reached a certain value; testing whether a queue is empty.

Condition Variables

- Each predicate should have its own unique condition variable.
- X • Sharing a single condition variable between more than one predicate can introduce inefficiency or errors unless you use extreme care.

Condition Variables

- Condition variables are typically used as a threesome:
 1. A boolean value that indicates whether the condition is met
 2. A mutex to serialize access to the boolean value
 3. A condition variable to wait for the condition to be met

接受方

```
mutex m = new mutex();
condition_variable cv = new
condition_variable();
bool desiredStateReached = false;
m.lock();
while (!desiredStateReached)
    cv.wait(m);
m.unlock();
```

通知方

```
m.lock();
desiredStateReached = true;
cv.signal();
m.unlock();
```

Implementing a Mutex

- If the lock is unlocked, the thread proceeds to lock it, executes the critical section, and unlocks it.
- If the lock is locked, the thread is put to sleep: the OS saves the entire thread context and pushes it out of the processor core into sleep queue. The processor core becomes available for use by another thread or process.
- The thread will leave the sleep queue and enter a runnable queue when the OS sees the lock unlocked (by another thread).
- When the thread is at the top of the runnable queue and core is available, it is de-queued and the context is loaded to the core.

Spinlock

- A spinlock is a “slim” implementation of a mutex whereby the thread does not go to sleep, but rather it actively spins for the lock to become unlocked.
- It consumes the processor core, but it does not incur the overhead of context saving and restoring.
- A spinlock may be more efficient than a mutex when a lock is only taken for a short period of time – e.g., incrementing a counter.



Complex Synchronization Objects

- Mutexes and condition variables can be used to build complex synchronization objects such as read-write locks and semaphores.

Example: Readers/Writers

In general, the number of reads to shared data exceeds the number of writes. Reading shared data concurrently can be permitted, but writing concurrently cannot be. Reading and writing concurrently cannot be permitted either.

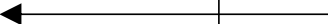
Using a simple mutex to protect shared data restricts concurrent read accesses as well. Design and implement (using pseudo-code) a ReadWriteLock object that allows concurrent read accesses, but disallows concurrent write or read/write accesses. You may use simple mutexes and/or condition variables in your code.

Hint. This object may have public methods readLock, readUnlock, writeLock, and writeUnlock to permit locking and unlocking under read and write accesses.

Example: Readers/Writers

```
class ReadWriteLock {  
private:  
    int noOfReaders; 读者数  
    mutex m;  
    condition_variable cv;  
public:  
    ReadWriteLock();  
    void readLock();  
    void readUnlock();  
    void writeLock();  
    void writeUnlock();  
};
```

Constructor: initializes *noOfReaders* to 0, and initializes the mutex *m*, and condition variable *cv*.



Example: Readers/Writers

```
void ReadWriteLock::readLock()
{
    m.lock();
    ++noOfReaders;
    m.unlock();
}

void ReadWriteLock::readUnlock()
{
    m.lock();
    --noOfReaders;
    if ( noOfReaders == 0 ) cv.signal(); // wake up a writer
    m.unlock();
}
```

Example: Readers/Writers

```
void ReadWriteLock::writeLock()
{
    m.lock();
    while ( noOfReaders > 0 ) cv.wait(m);
}
```

隐式释放

```
void ReadWriteLock::writeUnlock()
{
    cv.signal(); // wake up a writer
    m.unlock();
}
```

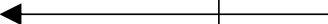
Semaphore 信号量

- Synchronization primitive to wait for a unit of resource to become available out of a maximum of N units.
 - Example: A store has three fitting rooms. You need exclusive access to any one room to try some clothes.
- A mutex provides a lock, a condition variable provides a wait queue, and a semaphore requires a counter in addition to a mutex and a condition variable.

Example: Semaphore

```
class Semaphore {  
private:  
    int count;  
    mutex m;  
    condition_variable cv;  
public:  
    Semaphore(int max);  
    void wait();  
    void signal();  
};
```

Constructor: sets *count* to max, and initializes the mutex *m*, and condition variable *cv*.



Example: Semaphore

```
void Semaphore::wait()
{
    m.lock();
    while(count <= 0)
        cv.wait(m);
    --count;
    m.unlock();
}
```

Example: Semaphore

```
void Semaphore::signal()
{
    m.lock();
    ++count;
    cv.signal();
    m.unlock();
}
```

Further Work

- Forgetting to unlock a mutex is likely to result in deadlocks.
Implement a lock (based on a mutex) that is self-unlocking when no longer in use.

= Using "

¿?

